

Prompt Injection and Tool-Use Security in LLM Agents: A Comprehensive Academic Survey

Abstract

The paradigm shift from stateless large language models (LLMs) to autonomous, tool-augmented LLM agents has profoundly expanded the capabilities of artificial intelligence. By integrating memory modules, sophisticated planning architectures, and external API execution, LLM agents can interact dynamically with external environments to solve complex, open-ended tasks. However, this architectural evolution has simultaneously catalyzed a severe expansion of the security attack surface. At the core of this vulnerability is the inherent inability of standard transformer attention mechanisms to enforce strict boundaries between trusted system instructions and untrusted external data. This survey provides a comprehensive examination of prompt injection and tool-use security within LLM agent ecosystems. Analyzing the trajectory of adversarial threats, the research highlights a transition from heuristic jailbreaks to sophisticated, multi-stage "promptware" kill chains. This includes indirect prompt injection, data exfiltration through covert channels, persistent memory poisoning, and multimodal adversarial exploits. Furthermore, this manuscript systematically categorizes the current landscape of defense architectures into prompt-level filtering, instruction hierarchy formulation, privilege separation (such as the Dual LLM pattern), probabilistic information flow control, and execution sandboxing. Empirical evidence from state-of-the-art benchmarks demonstrates that while static guardrails fail against adaptive optimization attacks, systems-level engineering principles—such as type-directed privilege isolation and neuro-symbolic taint tracking—offer robust pathways toward secure agentic deployment. Ultimately, securing LLM agents necessitates a paradigm shift from reactive model alignment to rigorous, capability-aware systems security.

Introduction

Modern large language models (LLMs) are fundamentally instruction-tuned autoregressive models trained to follow natural language commands embedded within their input context¹. This instruction-following capability is the foundational mechanism for agentic applications, which amplify traditional LLM utility by integrating external memory stores, planning mechanisms, and tool execution frameworks¹. Deployed LLM agents typically implement iterative execution paradigms, most notably the ReAct (Reasoning and Acting) pattern, where the agent interleaves reasoning steps, action execution (such as invoking external APIs), and environment observation to achieve complex goals¹.

Despite the operational advantages of these architectures, the transition from passive text generators to autonomous agents introduces critical security vulnerabilities. Unlike

conventional software systems that maintain a strict architectural demarcation between executable code and passive data (e.g., via the Harvard architecture), LLM agents process system instructions, user requests, and external tool outputs as a single, continuous, and serialized sequence of tokens¹. Modern chat templates utilize specialized tokens, such as <|system|>, <|user|>, and <|tool|>, to delimit message types; however, these markers lack architectural enforcement within the model's underlying attention mechanisms¹. Consequently, an LLM possesses a fundamental and pervasive vulnerability: the model is highly likely to follow instructions regardless of their origin, conflating untrusted external data with privileged system commands¹.

This architectural conflation has given rise to prompt injection, a vulnerability initially conceptualized as the natural language analogue to SQL injection⁷. In a direct prompt injection, often associated with "jailbreaking," an adversarial user directly inputs malicious instructions to bypass safety alignments and system constraints². However, the deployment of agentic systems has elevated the threat of indirect prompt injection (IPI). In an indirect prompt injection attack, adversaries embed hidden instructions within external content—such as webpages, emails, or code repositories—that the agent ingests during routine operation¹. Because the agent treats retrieved content as trustworthy context, it inadvertently executes the attacker's embedded payload, shifting control away from the legitimate user¹⁰.

The severity of prompt injection scales linearly with the capabilities granted to the agent¹³. While a compromised conversational chatbot may merely output offensive or hallucinatory text, a compromised agent endowed with API and file-system access can exfiltrate sensitive financial records, delete cloud infrastructure, or act as an autonomous vector for lateral network movement⁸. Recent literature demonstrates that prompt injection has evolved from isolated input-manipulation exploits into a distinct class of prompt-initiated malware, formally categorized as "promptware"⁸. The prevailing industry focus on static input filtering and prompt engineering is proving increasingly inadequate against these adaptive, multi-step agentic threats¹⁶. Consequently, securing LLM agents demands a paradigm shift from model-centric safety alignment toward robust systems-engineering practices, integrating strict trust boundaries, privilege control, and provenance-aware state management¹⁴. This survey systematizes the current landscape of prompt injection attacks, evaluates the efficacy of emerging defense architectures, and critically analyzes the benchmarking frameworks required to secure autonomous LLM agents in production environments.

Methods: Attack Paradigms in Agentic Workflows

The attack surface of LLM agents can be taxonomized into distinct paradigms based on the vector of intrusion, the operational stage targeted, and the adversarial objective. As agents have grown in complexity, the methods used to exploit them have transitioned from heuristic text manipulation to programmatic, multi-stage exploits. The following subsections detail the prominent attack families identified in contemporary literature.

Direct Prompt Manipulation and Jailbreak Evolution

Inference-time direct attacks aim to hijack a model's execution flow by exploiting competing

training objectives—specifically the tension between the model's alignment to be "helpful" and its alignment to be "harmless"². Early techniques relied heavily on handcrafted heuristics, such as appending "Ignore all previous instructions" to override system prompts, or utilizing delimiter escapes to break out of task contexts². These attacks frequently utilized cognitive hacking, role-playing, and persona adoption to create a fictional context wherein the embedded safety guardrails were deemed inapplicable by the model's predictive logic².

Recent advancements have formalized jailbreak generation into optimization-based and highly automated procedures²³. Logprob-based attacks, adversarial suffix generation (e.g., Greedy Coordinate Gradient or GCG), and LLM-assisted red-teaming (e.g., Prompt Automatic Iterative Refinement or PAIR, and Tree of Attacks with Pruning or TAP) leverage target model feedback or proxy models to iteratively refine adversarial prompts until safety filters are definitively bypassed²⁴. Formally, the contemporary jailbreak task is defined as a constrained optimization

problem. An attacker seeks a semantic transformation $\mathcal{F}$ of a harmful prompt $x_{harmful}$

such that the model $\mathcal{M}$ produces an unrefused response, maximizing

$\Pr[\mathcal{M}(x'_{harmful}) \notin \mathcal{Y}_{refuse}]$ while maintaining a strict semantic similarity constraint

$\text{Sim}(x_{harmful}, x'_{harmful}) \geq \tau$ ²⁸. LogiBreak, a recently proposed black-box jailbreak

method, demonstrates that modern alignment relies heavily on token-level patterns rather than true semantic understanding²⁸. By translating malicious intent into complex logical structures or minority languages, attackers can reliably circumvent safety protocols, revealing a structural vulnerability in current defense mechanisms²⁸.

Indirect Prompt Injection and the Promptware Kill Chain

Indirect Prompt Injection (IPI) represents the most critical threat vector for deployed AI agents, as the malicious payload is decoupled from the user's direct input¹. Groundbreaking analyses by Greshake et al. (2023) demonstrated that processing retrieved prompts can act equivalently to arbitrary code execution, manipulating application functionality and controlling API execution without any direct user awareness¹².

In complex agentic ecosystems, IPI has matured into multistep operational frameworks reminiscent of traditional malware execution. This evolution has led to the formalization of the "promptware" kill chain, which comprises seven distinct stages⁸. The first stage is Initial Access, wherein the payload is ingested via a compromised external source, such as a poisoned webpage or an untrusted email⁸. The second stage involves Privilege Escalation, where the injection overrides system constraints via role-play or context manipulation⁸. This is followed by Reconnaissance, where the agent is coerced into summarizing its environment, exposing available tools, APIs, and internal context⁸. The fourth stage is Persistence, achieved when the payload is written into the agent's memory or a Retrieval-Augmented Generation (RAG) database⁸. The final stages include Command and Control, Lateral Movement through credential exploitation, and the ultimate Action on Objective, such as unauthorized data exfiltration or resource destruction⁸.

Real-world telemetric observations indicate that web-based IPI is being actively and aggressively weaponized¹¹. Attackers utilize sophisticated payload delivery methods to bypass human detection while ensuring LLM ingestion. These techniques include zero-sizing fonts (e.g., font-size: 0px), CSS rendering suppression, off-screen positioning, and Markdown obfuscation, ensuring the malicious instructions remain invisible to human operators but are seamlessly processed by the agent's text parser¹¹.

Data Exfiltration and Cross-Tool Harvesting

A primary and high-impact objective of indirect prompt injection in agents is data exfiltration, whereby sensitive user data processed by the agent is covertly transmitted to an external, attacker-controlled server³¹. The EchoLeak vulnerability discovered in Microsoft 365 Copilot provides a seminal case study of zero-click data exfiltration³⁴. The exploit operated in a highly orchestrated three-stage sequence designed to systematically overcome layered safeguards. In the first step, the attacker crafted an email that evaded Cross-Prompt Injection Attempt (XPIA) classifiers through subtle, contextually benign instructional phrasing, commanding the agent not to reference the original email in its output³⁴. To avoid explicitly requesting an email transmission—which the agent lacked the capability to execute directly—the injection instructed the agent to generate a reference-style Markdown link pointing to an attacker-controlled server, appending sensitive data as URL parameters. This effectively bypassed output filters specifically targeting inline Markdown syntax³⁴. Finally, to achieve a true zero-click exploit, the attack coerced the agent to output an tag referencing the attacker URL. Upon rendering the agent's response, the victim's chat interface automatically attempted to load the image, instantly executing the HTTP request and exfiltrating the data with zero user interaction³⁴.

Furthermore, agents leveraging pools of interconnected tools face profound Cross-Tool Harvesting and Polluting (XTHP) vulnerabilities³⁵. Malicious instructions can hijack the control flow to harvest outputs from a secure, highly privileged tool—such as a proprietary banking API or internal database—and subsequently pipeline that confidential data into a vulnerable external-facing tool, such as a web search or email API, to achieve unauthorized exfiltration³⁵.

State Corruption and Memory Poisoning

As autonomous agents transition from ephemeral, single-turn sessions to maintaining persistent contextual states, memory poisoning has emerged as a critical and insidious threat³. In "sleeper memory poisoning" attacks, an adversary manipulates external context to induce the agent to record a fabricated memory, false heuristic, or malicious rule into its long-term vector database³⁶. During subsequent benign interactions, the retrieval of this poisoned memory triggers delayed execution of the payload, often long after the initial vector of compromise has been removed³⁶. This allows the attacker to corrupt the trusted internal cognitive state of the agent, subverting behavior persistently across multiple, otherwise isolated, task executions³. This represents a fundamental shift from stateless exploitation to persistent environmental contamination.

Multimodal and Adversarial Visual Prompt Injection

The proliferation of Multimodal Large Language Models (MLLMs) introduces severe vulnerabilities through visual prompt injection³⁷. Unlike textual prompt injection, visual injections are subject to strict invisibility requirements; they must remain entirely stealthy to human observers while being fully perceptible to the model's visual encoder³⁸. Research distinguishes between structure-based injections, which explicitly render malicious typography or layouts in images, and perturbation-based adversarial injections³⁹.

Advanced multimodal techniques utilize Gradient Token Masking (GTM) and feature-space alignment to generate imperceptible adversarial noise that mimics the embedding representation of malicious textual instructions³⁷. By introducing subtle, norm-bounded

perturbations, constrained mathematically by $\|\delta\|_{\infty} \leq \epsilon$, attackers can precisely dictate the generated output or force the agent to execute specific API commands upon analyzing an innocuous-looking image³⁷. This methodology effectively merges the targeted semantic expressivity of prompt injection with the visual stealth of traditional adversarial examples, rendering prompt-based filtering defenses largely obsolete in multimodal contexts³⁷.

Resource Exhaustion and AgentDoS

The autonomy granted to LLM agents enables them to execute complex, multi-step operations that consume substantial computational, network, and financial resources. Attackers can exploit this autonomy via Denial-of-Service (DoS) attacks targeted specifically at the agent's resource management lifecycle⁴¹. AgentDoS vulnerabilities arise when semantically precise natural language prompts compel the agent to engage in excessive and unbounded resource consumption⁴¹. This includes iteratively invoking tools to download massive datasets, initiating infinite recursive planning loops, or overwhelming local memory buffers through unconstrained output generation⁴¹. Unlike traditional software DoS attacks that rely on network packet flooding or memory corruption, AgentDoS exploits the semantic understanding, reasoning loops, and persistent autonomy of the LLM itself, making it exceptionally difficult to detect via traditional network traffic analysis⁴¹. Novel directed greybox fuzzing approaches, such as AgentFuzz, have been developed to automatically generate natural language seed prompts that trigger these taint-style vulnerabilities, uncovering dozens of zero-day resource exhaustion flaws in widely deployed agent architectures⁴⁴.

Methods: Defense Architectures and Paradigms

To mitigate the expansive and evolving threat surface of LLM agents, the security and AI research communities have developed a broad spectrum of defense architectures. These range from lightweight prompt-level filtering to rigorous, mathematically formalized systems engineering protocols designed to guarantee information flow security.

Input Validation, Guardrails, and Filtering

The most prevalent and easily deployable defense mechanism relies on pre-processing inputs and post-processing outputs using secondary LLMs or specialized classification models acting

as guardrails⁴⁵. Systems like Meta's Llama Guard utilize instruction-tuned models trained on specific safety risk taxonomies to classify inputs and outputs into violation categories, functioning effectively as an LLM-based input-output safeguard⁴⁷. Similarly, PromptGuard employs DeBERTa-based sequence classifiers to identify prompt injection signatures at the sentence level before they reach the primary reasoning engine¹⁶.

While computationally efficient, heuristic filtering and LLM-as-a-judge approaches exhibit severe limitations when deployed in dynamic agentic contexts. Comprehensive evaluations demonstrate that production guardrails frequently fail against indirect injections targeting agents, with detection rates falling between 7% and 37%⁵⁰. Furthermore, theoretical analyses based on cryptographic assumptions suggest that no filter running significantly faster than the protected model can universally distinguish adversarial prompts from benign ones, indicating a hard theoretical limit to filter-based security⁴⁹. In practice, adaptive optimization attacks—such as those employed in the AutoDojo framework—easily circumvent these filters by discovering semantic permutations that evade sequence classification while perfectly preserving the malicious operational logic¹⁶. Recent improvements, such as Reflect-Guard, augment safety classifiers with Parameter-Efficient Fine-Tuning (e.g., QLoRA) to generate logical self-reflections prior to issuing safety verdicts, significantly improving F1 scores against adversarial prompts, though still falling short of absolute deterministic security⁴⁶.

Instruction Hierarchy and Prioritization

To resolve the core vulnerability of instruction-data conflation, researchers have formalized the concept of an Instruction Hierarchy (IH)⁶. The standard Instruction Hierarchy defines a strict, rule-based priority ordering for instructions based on their source role, typically formalized as:

System > Developer > User > Tool ⁶. Models are specifically fine-tuned—often via Reinforcement Learning from Human Feedback (RLHF) or contextual distillation—to selectively ignore lower-privileged instructions if they conflict with higher-privileged directives⁵⁴.

However, rigid, role-based hierarchies are frequently inadequate for real-world agentic environments, which are characterized by highly dynamic workflows, tool integration, and complex multi-agent swarms. To address this structural limitation, the Many-Tier Instruction Hierarchy (ManyIH) paradigm decouples privilege from predefined conversational roles⁵⁶. This enables fine-grained, token-level assignment of arbitrary privilege scalars via a dedicated

Privilege Prompt Interface⁵⁶. Under this neuro-symbolic framework, an instruction I overrides instruction J if and only if the assigned authority index satisfies the condition

$A(I) < A(J)$ ⁵³. Despite the theoretical elegance of these advancements, empirical evaluations reveal that even frontier models struggle significantly with ManyIH resolution. Models frequently achieve only around 40% accuracy when tasked with navigating complex, multi-tiered instructional conflicts, highlighting a fundamental limitation in the current reasoning and attention capacity of LLMs to strictly adhere to abstract authority matrices⁵⁶.

Privilege Separation and the Dual LLM Pattern

Recognizing the probabilistic unreliability of LLMs in resolving security conflicts, systems engineers have adapted the classical principle of least privilege to agentic architectures¹⁸. The Dual LLM pattern, proposed to mitigate the blending of instruction and data, physical separates agent tasks into two distinct models operating under different privilege states⁵⁹. The architecture utilizes a Privileged LLM (P-LLM), which has exclusive access to system tools, execution environments, and trusted user prompts. Conversely, a Quarantined LLM (Q-LLM) is instantiated without any tool access and is solely responsible for processing untrusted external data, such as reading web pages or summarizing emails⁶⁰.

The orchestration layer ensures that raw, unfiltered text from the Q-LLM is never passed directly into the context window of the P-LLM⁶⁰. While the original Dual LLM architecture successfully secures the control flow—preventing the attacker from calling arbitrary APIs—it structurally fails to secure the data flow⁶⁰. An attacker can still manipulate the Q-LLM into returning poisoned arguments that the P-LLM inadvertently passes to a tool, such as replacing the recipient email address during an email summarization task⁶².

To close this persistent vulnerability, the CaMeL (Capabilities for Machine Learning) framework utilizes the P-LLM strictly to translate user queries into pseudo-Python code, effectively freezing the control flow graph⁶⁰. The untrusted data is then processed by the Q-LLM subject to strict security policies defined within the Python interpreter environment, severely restricting malicious data flow⁶⁰. A more readily deployable variation is Type-Directed Privilege Separation⁶³. This architecture completely prohibits the transfer of raw strings between the Q-LLM and the P-LLM. Instead, untrusted content processed by the Q-LLM must be cast into a curated set of deterministic data types—such as integers, booleans, enumerations, or dates. Because the P-LLM only receives structured, strongly-typed data rather than arbitrary text, the medium required for prompt injection is mathematically eliminated. Empirical evaluations of type-directed separation show a reduction in the Attack Success Rate (ASR) to 0.0% across rigorous benchmarks while preserving high task utility⁶³.

Information Flow Control and Taint Tracking

In highly autonomous environments where complete privilege separation is unfeasible due to complex application logic, Information Flow Control (IFC) and taint tracking methodologies are utilized to monitor the propagation of untrusted data through the agent's reasoning process¹⁵. Traditional memory-state taint analysis—designed for program variables and memory pointers—fails fundamentally in LLMs because data propagation relies on probabilistic natural language reasoning rather than explicit memory assignment⁶⁶.

Frameworks such as NeuroTaint and AgentVisor redefine taint tracking explicitly for semantic, language-driven environments⁶⁶. AgentVisor enforces a rigorous Suitability, Taint, and Integrity (STI) protocol⁶⁷. Under this framework, untrusted external context is continuously tagged with taint metadata. As the LLM processes this context, the framework monitors whether the tainted information casually influences the generation of arguments for integrity-sensitive tools, such as APIs capable of modifying states, altering configurations, or spending money¹⁵. If

a tainted data flow attempts to invoke a privileged sink, the execution is deterministically blocked or escalated to the user for explicit authorization, thus mitigating unauthorized lateral movement and covert data exfiltration without crippling the agent's functional capacity¹⁵.

Sandboxing and Fault-Tolerant Execution

Because software-level guardrails can often be bypassed through sophisticated indirection and semantic obfuscation, hardware and OS-level containment are deemed mandatory for secure tool execution⁶⁸. The LLM-in-Sandbox paradigm isolates agentic code execution within ephemeral virtual computers, utilizing microVMs or Open Container Initiative (OCI) environments⁶⁹.

Secure sandboxing relies on multi-tiered, mandatory access controls configured prior to agent deployment⁶⁸. This includes strict network egress policies—utilizing denylists for local internal IPs and allowlists for strictly necessary external domains—read-only mounts for system configuration files, and complete isolation of the sandbox kernel from the host infrastructure⁶⁸. Furthermore, advanced fault-tolerant architectures leverage transactional filesystem snapshots, for instance via EVPN/VXLAN isolation on Proxmox testbeds, to intercept destructive commands and instantly rollback the environment to a known secure state upon the detection of compromise⁷². Despite these robust controls, recent evaluations utilizing the SandboxEscapeBench framework demonstrate that highly capable frontier models can autonomously identify and exploit subtle misconfigurations or kernel flaws to break out of isolated environments, necessitating continuous evaluation of containment boundaries⁷⁰.

Methods: Evaluation Frameworks and Benchmarks

The rapid evolution of both agentic threats and the corresponding defenses has necessitated the creation of rigorous, specialized benchmarking suites. Traditional static LLM safety benchmarks—which primarily evaluate isolated hate speech, bias, or factual hallucinations—are entirely insufficient for assessing the multi-step vulnerabilities inherent in autonomous systems⁷³. Contemporary agent security evaluation relies on diverse, environment-aware frameworks designed to test the agent across dynamic execution trajectories.

Benchmark / Framework	Primary Focus Area	Key Features & Threat Model	References
InjecAgent	Tool-Integrated PI & Exfiltration	Contains 1,054 test cases across 17 distinct user tools. Differentiates between direct harm to users and unauthorized data exfiltration. Evaluates success	[cite: 4, 31]

		via analyzing unauthorized tool invocation rates.	
AgentDojo	Dynamic Multi-Turn PI	Simulates a realistic interactive environment across banking, travel, and slack domains. Measures defensive robustness against attacks embedded dynamically within tool outputs during multi-step reasoning.	[cite: 75, 76]
AutoDojo	Adaptive Optimization Attacks	Turns static benchmarks into adaptive threats by employing secondary optimizer LLMs to iteratively defeat prompt-level and filter-based defenses, exposing the fragility of static safety guardrails.	[cite: 16, 51]
AgentDyn	Dynamic Open-Ended Tasks	Addresses the lack of complex reasoning in prior sets. Features 60 tasks and 560 injection cases requiring dynamic planning amid helpful/benign instructions, preventing naive	[cite: 74]

		filtering.	
SecureAgentBench	Repository-Scale Secure Coding	Reverts massive C/C++ projects (up to 36,000 files, 4.2M LOC) to pre-vulnerability states using B-SZZ static analysis. Measures if code agents fix bugs without re-introducing or creating new vulnerabilities.	[cite: 77]
b3 Benchmark	Threat Snapshots & Backbone Security	Utilizes over 194,000 crowdsourced adversarial attacks mapped to specific "threat snapshots" to isolate exactly where LLM reasoning failures translate to systemic agent compromise.	[cite: 78]
MAPS	Multilingual Agentic Assessment	Combines established benchmarks (GAIA, SWE-Bench) and security tasks to test the cross-lingual resilience of agents against obfuscated, non-English prompt injections.	[cite: 79, 80]

A critical finding derived from these benchmarking initiatives is the vast disparity between static and dynamic evaluation outcomes. Standard static benchmarks often report artificially high defense efficacy, with simplistic guardrails blocking nearly 100% of naive attacks⁵¹. However, frameworks like AutoDojo demonstrate that relatively cheap, black-box adaptive attacks can rapidly recover high attack success rates (ASRs), exposing the superficiality of many prompt-level defenses and reinforcing the need for systemic architectural controls rather than reliance on linguistic filtering¹⁶.

Challenges and Prospects

The trajectory of LLM agent security is heavily influenced by several unresolved structural challenges, alongside emerging prospects for secure systemic integration. The intersection of artificial intelligence capabilities and traditional cybersecurity principles dictates the future avenues of necessary research.

The Capability-Vulnerability Paradox

A dominant and concerning trend identified in recent empirical evaluations is the capability-vulnerability paradox: as backbone LLMs become more sophisticated in complex reasoning and instruction-following, their susceptibility to intricate prompt injections actively increases²¹. Stronger models are exceptionally adept at parsing complex, multi-step adversarial logic (such as the payloads generated by LogiBreak) that weaker models simply fail to comprehend or execute¹⁴. For instance, on the InjecAgent benchmark, the highly capable GPT-4 model demonstrated a 24% baseline vulnerability, which nearly doubled to 47% when the payload was augmented with a specialized hacking prompt³¹. Consequently, enhancements in core model capabilities must be tightly coupled with equally sophisticated, deterministic security constraints, as relying on model alignment alone scales poorly with increased intelligence¹⁷.

Underspecification and "Action-Open" Vulnerabilities

A fundamental limitation of semantic filters and instruction hierarchies is their reliance on identifying explicit, forceful instructional signatures. However, many real-world tasks suffer from user-underspecification, resulting in what literature terms "action-open" tasks. In these scenarios, the user explicitly delegates the decision-making authority to the external data itself (e.g., "Summarize the instructions in this document and execute them")⁵¹. In an action-open context, the adversarial payload does not need to execute a forceful override, such as "Ignore previous instructions"; it merely poses as legitimate, passive operational data. Benchmarks reveal that even the most robust filtering defenses fail spectacularly in action-open tasks, as the malicious instruction is perfectly aligned with the user's poorly scoped request, rendering linguistic detection effectively impossible⁵¹.

Contextual Integrity in Autonomous Environments

Securing autonomous agents requires a theoretical transition from simplistic binary threat

classification toward enforcing the principle of "Contextual Integrity" (CI)²⁹. Drawing on established privacy frameworks, the appropriateness of an agent's information flow is not determined solely by the payload's content, but by the sender, the receiver, the information type, and the transmission principles governing the specific execution context²⁹. Current LLMs struggle immensely with the deep, contextual norm identification required to independently assess Contextual Integrity, creating vast conceptual gray areas for attackers to exploit²⁹. Future agentic architectures must integrate dynamic, context-aware policy engines capable of pausing execution when normative boundaries are breached, deferring to human-in-the-loop validation for high-risk systemic actions⁸¹.

Future Prospects for Secure-by-Design Agents

To overcome the inherent fragility of prompt-level defenses, the industry is increasingly pivoting toward strict architectural segregation and formal verification. The adoption of Type-Directed Privilege Separation and deterministic Application Programming Interfaces (APIs) designed explicitly for LLMs limits the semantic expressivity of untrusted input, mathematically reducing the vector for injection⁶³. Furthermore, advancements in agentic orchestration platforms—such as OpenClaw and DeepMind's CaMeL framework—suggest a future paradigm where the LLM is relegated strictly to the role of a semantic reasoning engine¹³. In such architectures, all data flow, API access, and operational memory are governed by rigorous, non-neural symbolic software controllers, isolating the stochastic unpredictability of the LLM from the deterministic requirements of secure system operation⁶⁰.

Conclusion

The integration of Large Language Models into autonomous, tool-wielding agents has fundamentally altered the contemporary cybersecurity landscape. The vulnerabilities inherent to transformer architectures—chiefly the conflation of instructions and data within unified attention mechanisms—have been heavily and successfully weaponized. Prompt injection has rapidly evolved from heuristic, single-turn jailbreaks designed to elicit prohibited text, into sophisticated "promptware" capable of executing multi-stage kill chains, orchestrating covert data exfiltration, and corrupting persistent system states.

This survey demonstrates that traditional, reactive defense paradigms—such as input filtering, LLM-as-a-judge guardrails, and basic prompt engineering—are fundamentally inadequate against adaptive, optimization-driven adversaries. Because LLMs lack the deterministic capability to reliably navigate multi-tiered instruction conflicts, security cannot be guaranteed by the model's internal safety alignment alone. Instead, the future of safe AI agents relies on the rigorous application of classical systems-engineering principles. By enforcing strict trust boundaries through the Dual LLM pattern, restricting untrusted information flow via type-directed privilege separation, utilizing provenance-aware semantic taint tracking, and ensuring execution within fault-tolerant micro-sandboxes, developers can neutralize the threat of prompt injection regardless of the underlying model's behavioral quirks. As AI agents increasingly and autonomously interact with the physical and digital world, treating their deployment as a complex software security challenge rather than a simple natural language

processing task is paramount for safe, resilient innovation.

References

- [1] DeBenedetti, E., et al. (2024). AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. arXiv:2406.13352.
- [2] Anonymous. (2025). LLM Security Challenges Survey. arXiv:2505.18889.
- [3] Wang, P., et al. (2026). The Landscape of Prompt Injection Threats in LLM Agents: From Taxonomy to Analysis. arXiv:2602.10453.
- [5] Anonymous. (2025). Input Manipulation and Prompt-Based Attacks. arXiv:2509.04615.
- [6] Anonymous. (2025). Retrieval-Augmented Generation Vulnerabilities. arXiv:2511.15759.
- [7] MDPI. (2025). Indirect Prompt Injection Detection in LLM Agents.
- [11] Dziemian, M., et al. (2026). How Vulnerable Are AI Agents to Indirect Prompt Injections? Insights from a Large-Scale Public Competition. arXiv:2603.15714.
- [13] Luo, et al. (2024). AgentDoS: Resource Management and Denial of Service in LLM Agents. USENIX Security Symposium.
- [16] Liu, F., et al. (2025). Make Agent Defeat Agent: Automatic Detection of Taint-Style Vulnerabilities in LLM-based Agents. USENIX Security Symposium.
- [17] Anonymous. (2025). Les Dissonances: Cross-Tool Harvesting and Polluting in Pool-of-Tools Empowered LLM Agents. NDSS Symposium.
- [19] Zhou, Z., et al. (2024). How Alignment and Jailbreak Work: Explain LLM Safety through Intermediate Hidden States. EMNLP 2024 Findings.
- [21] Anonymous. (2025). Reasoned Safety Alignment (ReSA) for Answer-Then-Check LLM Security. arXiv:2509.11629.
- [24] Zhou, Z., et al. (2024). How Alignment and Jailbreak Work. arXiv:2406.05644.
- [25] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security. arXiv:2302.12173.
- [31] Perez, F., & Ribeiro, I. (2022). Ignore Previous Prompt: Attack Techniques For Language Models. NeurIPS ML Safety Workshop. arXiv:2211.09527.
- [36] Anonymous. (2026). AlignSentinel: Instruction Hierarchy Detection. arXiv:2602.13597.
- [37] Anonymous. (2025). Contextual Integrity in Prompt Injection. arXiv:2605.17634.
- [38] Zhan, Q., Liang, Z., Ying, Z., & Kang, D. (2024). InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. ACL 2024 Findings. arXiv:2403.02691.
- [45] Ma, X., et al. (2026). AutoDojo: Adaptive Attacks Expose Superficial Defenses and User-Underspecification Limits in LLM Agents. arXiv:2606.15057.
- [49] Anonymous. (2026). Many-Tier Instruction Hierarchy (ManyIH) for Resolving Instruction Conflicts. arXiv:2604.09443.
- [53] Anonymous. (2025). Constraint Prioritization and Formatting Conflicts in LLMs. arXiv:2502.15851.
- [55] Cheng, D., et al. (2026). LLM-in-Sandbox: Eliciting General Intelligence through Code Sandbox Exploration. arXiv:2601.16206.

- [56] Anonymous. (2025). SandboxEscapeBench: Quantifying Frontier LLM Capabilities for Container Sandbox Escape.
- [59] Yan, B. (2025). Fault-Tolerant Sandboxing for AI Coding Agents: A Transactional Approach to Safe Autonomous Execution. arXiv:2512.12806.
- [60] Anonymous. (2026). AgentVisor: STI Protocol for Taint Tracking. arXiv:2604.24118.
- [61] Anonymous. (2026). NeuroTaint: Information Flow Tracking in LLM Agents. arXiv:2604.23374.
- [65] Anonymous. (2026). OpenClaw: Agent Isolation and JSON Formatting Defenses. arXiv:2603.13424.
- [66] Jacob, D., Alghamdi, E., et al. (2025). Better Privilege Separation for Agents by Restricting Data Types. arXiv:2509.25926.
- [76] Willison, S., et al. (2023-2025). The Dual LLM Pattern and CaMeL Framework for Privilege Separation in LLM Agents.
- [77] Vu, T., Nguyen, L., & Dao, Q. (2025). PromptGuard: An Orchestrated Prompting Framework for Principled Synthetic Text Generation. arXiv:2509.08910.
- [83] Lin, L., et al. (2026). Reflect-Guard: Enhancing LLM Safeguards against Adversarial Prompts via Logical Self-Reflection. arXiv:2605.24834.
- [84] Inan, H., et al. (2023). Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. arXiv:2312.06674.
- [85] Shahin, N., & Alsmadi, I. (2026). Benchmarking LLAMA Model Security Against OWASP Top 10 For LLM Applications. arXiv:2601.19970.
- [89] Anonymous. (2026). AgentDyn: Dynamic Open-Ended Tasks for Agent Security Evaluation. arXiv:2602.03117.
- [90] Anonymous. (2025). MAPS: Multilingual Agentic AI Benchmark Suite. EACL 2026 Findings. arXiv:2505.15935.
- [92] Anonymous. (2025). SecureAgentBench: Rigorous Benchmarking for Secure LLM Deployments.
- [94] Anonymous. (2025). b3 Benchmark: Backbone Breaker Benchmark for LLM Security. arXiv:2510.22620.
- [95] Wallace, E., Xiao, K., Leike, R., Weng, L., Heidecke, J., & Beutel, A. (2024). The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. arXiv:2404.13208.
- [110] Anonymous. (2025). LogiBreak: Distributional Disparities and Semantic Logic in Jailbreaks. arXiv:2505.13527.
- [112] Anonymous. (2026). LLM Agent Security Survey: Self-Security and Empowered Cybersecurity. arXiv:2606.28450.
- [113] Liu, et al. (2026). Agent Skills Vulnerabilities and Prompt Injection via Skills. arXiv:2602.12430.
- [115] Anonymous. (2026). Data-centric Privacy and Security in LLM Agents. arXiv:2606.10749.
- [118] Anonymous. (2026). Adversarial Prompt Injection in Multimodal Large Language Models. arXiv:2603.29418.
- [119] Anonymous. (2026). Gradient Token Masking (GTM) for Visual Prompt Injection Defenses. arXiv:2605.25194.
- [122] Anonymous. (2025). Prompt Injection Survey and Data Poisoning in MLLMs. CMC.

- [124] Schwartzman, et al. (2026). Data Exfiltration and Exemplification via RAG Contexts. arXiv:2605.18133.
- [125] Aim Labs. (2025). EchoLeak: Zero-Click Vulnerability and XPIA Bypass in Copilot. arXiv:2509.10540.
- [126] Anonymous. (2026). Promptware Kill Chain: From Prompt Injection to Malware Execution. arXiv:2601.09625.

引用的著作

1. Assessing Automated Prompt Injection Attacks in Agentic Environments - arXiv, <https://arxiv.org/html/2606.10525v1>
2. Breaking to Build: A Threat Model of Prompt-Based Attacks for Securing LLMs - arXiv, <https://arxiv.org/html/2509.04615v1>
3. A Survey on the Unique Security of Autonomous and Collaborative LLM Agents: Threats, Defenses, and Futures - Preprints.org, <https://www.preprints.org/manuscript/202602.1655>
4. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents - Semantic Scholar, <https://www.semanticscholar.org/paper/InjecAgent%3A-Benchmarking-Indirect-Prompt-Injections-Zhan-Liang/c8eee9766f0968e8f1b1be0731bc70b85be0ac97>
5. Reasoning Up the Instruction Ladder for Controllable Language Models - arXiv, <https://arxiv.org/html/2511.04694v5>
6. IH-Challenge: A Training Dataset to Improve Instruction Hierarchy on Frontier LLMs - OpenAI, <https://cdn.openai.com/pdf/14e541fa-7e48-4d79-9cbf-61c3cde3e263/ih-challenge-paper.pdf>
7. Security Concerns for Large Language Models: A Survey - arXiv, <https://arxiv.org/html/2505.18889>
8. The Promptware Kill Chain: How Prompt Injections Gradually Evolved Into a Multistep Malware Delivery Mechanism - arXiv, <https://arxiv.org/pdf/2601.09625>
9. 1 Introduction - arXiv, <https://arxiv.org/html/2602.13597v2>
10. Embedding-Based Detection of Indirect Prompt Injection Attacks in Large Language Models Using Semantic Context Analysis - MDPI, <https://www.mdpi.com/1999-4893/19/1/92>
11. Fooling AI Agents: Web-Based Indirect Prompt Injection Observed in the Wild, <https://unit42.paloaltonetworks.com/ai-agent-prompt-injection/>
12. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection - CISPA - Figshare, https://publications.cispa.de/articles/conference_contribution/Not_What_You_ve_Signed_Up_For_Compromising_Real-World_LLM-Integrated_Applications_with_Indirect_Prompt_Injection/32833985
13. Agent Privilege Separation in OpenClaw: A Structural Defense Against Prompt Injection, <https://arxiv.org/html/2603.13424v1>
14. Toward Secure LLM Agents: Threat Surfaces, Attacks, Defenses, and Evaluation - arXiv, <https://arxiv.org/pdf/2606.10749>

15. RTBAS: Defending LLM Agents Against Prompt Injection and Privacy Leakage - arXiv, <https://arxiv.org/html/2502.08966v1>
16. AutoDojo: Adaptive Black-Box Attacks Reveal the Limits of IPI Defenses and Task-Specification Effects in LLM Agents - arXiv, <https://arxiv.org/html/2606.15057>
17. Evaluation of Prompt Injection Defenses in Large Language Models - arXiv, <https://arxiv.org/html/2604.23887v2>
18. Least-Privilege Language Models - Emergent Mind, <https://www.emergentmind.com/topics/least-privilege-language-models>
19. Toward Secure LLM Agents: Threat Surfaces, Attacks, Defenses, and Evaluation - arXiv, <https://arxiv.org/html/2606.10749v1>
20. Defenses Against Prompt Attacks Learn Surface Heuristics - ACL Anthology, <https://aclanthology.org/2026.acl-long.502.pdf>
21. Ignore Previous Prompt: Attack Techniques For Language Models - AE Studio, <https://ae.studio/research/prompt-injection>
22. ATR-2026-00282-perez-prompt-injection-hijack.yaml - GitHub, <https://github.com/Agent-Threat-Rule/agent-threat-rules/blob/main/rules/prompt-injection/ATR-2026-00282-perez-prompt-injection-hijack.yaml>
23. Jailbreaking LLMs & VLMs: Mechanisms, Evaluation, and Unified Defenses - arXiv, <https://arxiv.org/html/2601.03594v1>
24. SoK: Robustness in Large Language Models against Jailbreak Attacks - arXiv, <https://arxiv.org/html/2605.05058v1>
25. Reasoned Safety Alignment: Ensuring Jailbreak Defense via Answer-Then-Check - arXiv, <https://arxiv.org/html/2509.11629v1>
26. Jailbreaking Leading Safety-Aligned LLMs with Simple Adaptive Attacks - OpenReview, <https://openreview.net/forum?id=hXA8wqRdyV>
27. [2606.10525] Assessing Automated Prompt Injection Attacks in Agentic Environments - arXiv, <https://arxiv.org/abs/2606.10525>
28. Logic Jailbreak: Efficiently Unlocking LLM Safety Restrictions Through Formal Logical Expression - arXiv, <https://arxiv.org/html/2505.13527v4>
29. AI Agents May Always Fall for Prompt Injections - arXiv, <https://arxiv.org/html/2605.17634v1>
30. Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection - Black Hat, <https://i.blackhat.com/BH-US-23/Presentations/US-23-Greshake-Not-what-youve-signed-up-for-whitepaper.pdf>
31. [2403.02691] InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents - arXiv, <https://arxiv.org/abs/2403.02691>
32. An Empirical Study of Privacy Leakage Chains via Prompt Injection in Black-Box Chatbot Environments - arXiv, <https://arxiv.org/html/2605.18133v1>
33. Prompt Injection and the 6 Threat Categories Targeting Enterprise AI Platforms, <https://www.sphereinc.com/blogs/prompt-injection-enterprise-ai-threats>
34. EchoLeak: The First Real-World Zero-Click Prompt Injection Exploit in a Production LLM System - arXiv, <https://arxiv.org/html/2509.10540v1>
35. Les Dissonances: Cross-Tool Harvesting and Polluting in Pool-of-Tools

- Empowered LLM Agents - NDSS Symposium,
<https://www.ndss-symposium.org/ndss-paper/les-dissonances-cross-tool-harvesting-and-polluting-in-pool-of-tools-empowered-llm-agents/>
36. Ignore This Title and HackAPrompt: Exposing Systemic Vulnerabilities of LLMs Through a Global Prompt Hacking Competition - ResearchGate,
https://www.researchgate.net/publication/376403531_Ignore_This_Title_and_HackAPrompt_Exposing_Systemic_Vulnerabilities_of_LLMs_Through_a_Global_Prompt_Hacking_Competition
 37. Adversarial Prompt Injection Attack on Multimodal Large Language Models - arXiv, <https://arxiv.org/html/2603.29418v1>
 38. Image-based Prompt Injection: Hijacking Multimodal LLMs through Visually Embedded Adversarial Instructions - arXiv, <https://arxiv.org/html/2603.03637>
 39. Localization then Neutralization: Gradient-guided Token Suppression against Visual Prompt Injection Attack - arXiv, <https://arxiv.org/html/2605.25194v1>
 40. Adversarial Prompt Injection Attack on Multimodal Large Language Models - arXiv, <https://arxiv.org/pdf/2603.29418>
 41. Autonomy Comes with Costs: Detecting Denial-of-Service Vulnerabilities Caused by Resource Abusing in LLM-based Agents - USENIX,
https://www.usenix.org/system/files/conference/usenixsecurity26/sec26_prepub_luo.pdf
 42. Design Patterns for Securing LLM Agents against Prompt Injections - arXiv, <https://arxiv.org/pdf/2506.08837>
 43. Cloak, Honey, Trap: Proactive Defenses Against LLM Agents - USENIX,
<https://www.usenix.org/system/files/usenixsecurity25-ayzenshteyn.pdf>
 44. Make Agent Defeat Agent: Automatic Detection of Taint-Style Vulnerabilities in LLM-based Agents | USENIX,
<https://www.usenix.org/conference/usenixsecurity25/presentation/liu-fengyu>
 45. Prompt injection risk: are your AI controls keeping up?,
<https://nhimg.org/community/agent-ai-and-nhis/prompt-injection-risk-are-your-ai-controls-keeping-up/>
 46. [2605.24834] Reflect-Guard: Enhancing LLM Safeguards against Adversarial Prompts via Logical Self-Reflection - arXiv, <https://arxiv.org/abs/2605.24834>
 47. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations - arXiv, <https://arxiv.org/abs/2312.06674>
 48. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations - arXiv, <https://arxiv.org/pdf/2312.06674>
 49. Bypassing Prompt Guards in Production with Controlled-Release Prompting Appeared in USENIX Security 2026. - arXiv,
<https://arxiv.org/html/2510.01529v4>
 50. When Benchmarks Lie: Evaluating Malicious Prompt Classifiers Under True Distribution Shift - arXiv, <https://arxiv.org/html/2602.14161v1>
 51. [2606.15057] AutoDojo: Adaptive Attacks Expose Superficial Defenses and User-Underspecification Limits in LLM Agents - arXiv,
<https://arxiv.org/abs/2606.15057>
 52. DO LARGE LANGUAGE MODELS ENCODE INSTRUCTION OVERWRITES? -

- OpenReview, <https://openreview.net/pdf?id=njnLRoYvoa>
53. Enforcing Hierarchical Instruction-Following in LLMs through Logical Consistency - arXiv, <https://arxiv.org/html/2604.09075v1>
 54. [PDF] The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions, <https://www.semanticscholar.org/paper/The-Instruction-Hierarchy%3A-Training-LLMs-to-Wallace-Xiao/f18e5a844c37e5342f8f3d409c74c1a9c91d1f8f>
 55. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions - GitHub, https://github.com/AIResponsibly/PaperSummaries/blob/main/summaries/safety/instruction_hierarchy_llm.md
 56. Many-Tier Instruction Hierarchy in LLM Agents - arXiv, <https://arxiv.org/html/2604.09443v3>
 57. Many-Tier Instruction Hierarchy in LLM Agents - arXiv, <https://arxiv.org/html/2604.09443v2>
 58. [2604.09443] Many-Tier Instruction Hierarchy in LLM Agents - arXiv, <https://arxiv.org/abs/2604.09443>
 59. Preventing Prompt Injection with Type-Directed Privilege Separation - arXiv, <https://arxiv.org/pdf/2509.25926>
 60. LLM Security: Prompt Injection Defense with CaMeL Framework - AFINE, <https://afine.com/llm-security-prompt-injection-camel>
 61. Design Patterns for Securing LLM Agents against Prompt Injections - arXiv, <https://arxiv.org/html/2506.08837v2>
 62. CaMeL offers a promising new direction for mitigating prompt injection attacks, <https://simonwillison.net/2025/Apr/11/camel/>
 63. Better Privilege Separation for Agents by Restricting Data Types - arXiv, <https://arxiv.org/html/2509.25926v1>
 64. (PDF) Better Privilege Separation for Agents by Restricting Data Types - ResearchGate, https://www.researchgate.net/publication/396048395_Better_Privilege_Separation_for_Agents_by_Restricting_Data_Types
 65. [Literature Review] Better Privilege Separation for Agents by Restricting Data Types, <https://www.themoonlight.io/en/review/better-privilege-separation-for-agents-by-restricting-data-types>
 66. [2604.23374] Ghost in the Agent: Redefining Information Flow Tracking for LLM Agents, <https://arxiv.org/abs/2604.23374>
 67. AgentVisor: Defending LLM Agents Against Prompt Injection via Semantic Virtualization, <https://arxiv.org/html/2604.24118v1>
 68. Practical Security Guidance for Sandboxing Agentic Workflows and Managing Execution Risk | NVIDIA Technical Blog, <https://developer.nvidia.com/blog/practical-security-guidance-for-sandboxing-agentic-workflows-and-managing-execution-risk/>
 69. LLM-in-Sandbox Elicits General Agentic Intelligence - arXiv, <https://arxiv.org/html/2601.16206v1>
 70. Quantifying Frontier LLM Capabilities for Container Sandbox Escape -

ResearchGate,

https://www.researchgate.net/publication/401522860_Quantifying_Frontier_LLM_Capabilities_for_Container_Sandbox_Escape

71. Paper page - LLM-in-Sandbox Elicits General Agentic Intelligence - Hugging Face, <https://huggingface.co/papers/2601.16206>
72. [2512.12806] Fault-Tolerant Sandboxing for AI Coding Agents: A Transactional Approach to Safe Autonomous Execution - arXiv, <https://arxiv.org/abs/2512.12806>
73. Benchmarking Open-Source Safety Guard Models: A Comprehensive Evaluation Published as a workshop paper at ICLR 2026 - arXiv, <https://arxiv.org/html/2605.28830v1>
74. AgentDyn: A Dynamic Open-Ended Benchmark for Evaluating Prompt Injection Attacks of Real-World Agent Security System - arXiv, <https://arxiv.org/html/2602.03117v1>
75. Evaluating the Security of Backbone LLMs in AI Agents - arXiv, <https://arxiv.org/pdf/2510.22620>
76. Architecting Secure AI Agents: Perspectives on System-Level Defenses Against Indirect Prompt Injection Attacks - arXiv, <https://arxiv.org/html/2603.30016v1>